

VNR Vignana Jyothi Institute of Engineering and Technology

(Affiliated to JNTU, Hyderabad)

Bachupally(v), Hyderabad, Telangana, India.

RESUME SKILL GAP ANALYSER

A COURSE BASED PROJECT REPORT SUBMITTED IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE COMPLETION OF COURSE

"Object Oriented Programming Through Java Laboratory"

BACHELOR OF TECHNOLOGY

IN

CSE - AIML

Submitted by

Bandi Sidhartha Reddy	25071A6679
Borra Pavan Sai Abhishek	25071A6683
Nagothu Mithesh	25071A66B5
Sindaram Saathwik Reddy	25071A66D4

Under the guidance of

Ms. K. Anitha

Assistant Professor

Department of Computer Science & Engineering (AI & ML) & IoT and R&AI



VNR Vignana Jyothi Institute of Engineering and Technology

(Affiliated to JNTU, Hyderabad)

Bachupally(v), Hyderabad, Telangana, India.

CERTIFICATE

This is to certify that they completed their course project work at Department of Computer Science & Engineering (AI & ML) & IoT and R&AI of VNR VJIET, Hyderabad entitled "Resume Skill Gap Analyser" in complete fulfilment of the requirements for the award of B. Tech degree during the academic year 2026-2027. This work is carried out under my supervision and has not been submitted to any other University/Institute for award of any degree/diploma.

Signature of Supervisor

Ms. K. Anitha

Assistant Professor

Dept. of CSE - (AI & ML) & IoT and R&AI

VNRVJIET, Hyderabad

Signature of HOD

Dr. Sagar Yeruva

Assoc. Professor & HOD

Dept. of CSE - (AI & ML) & IoT and R&AI

VNRVJIET, Hyderabad

DECLARATION

This is to certify that our project report titled “Resume Skill Gap Analyser” submitted to Vallurupalli Nageswara Rao Institute of Engineering and Technology in complete fulfilment of requirement for the award of Bachelor of Technology in Computer Science and Engineering (AI & ML) & IoT and R&AI is a Bonafide report to the work carried out by us under the guidance and supervision of Ms. K. Anitha, Assistant Professor, Department of CSE (AI & ML) & IoT and R&AI, Vallurupalli Nageswara Rao Institute of Engineering and Technology. To the best of our knowledge, this has not been submitted in any form to other universities or institutions for the award of any degree or diploma.

Bandi Sidhartha Reddy
25071A6679
CSE – AIML

Borra Pavan Sai Abhishek
25071A6683
CSE - AIML

Nagothu Mithesh
25071A66B5
CSE – AIML

Sindaram Saathwik Redy
25071A66D4
CSE - AIML

ACKNOWLEDGEMENT

Over the last one year, VNRVJIET has helped us transform ourselves from mere amateurs in the field of Computer Science into skilled engineers capable of handling given situations in real time. We are highly indebted to the institute for everything that it has given us. We would like to express our gratitude towards the principal of our institute, Dr. B. Chennakesava Rao and the Head of the Department, Computer Science & Engineering (AI & ML) & IoT and R&AI, Dr. Y. Sagar, for their kind co-operation and encouragement which helped us complete the project in the stipulated time. Although we have spent a lot of time and put in a lot of effort into this project, it would not have been possible without the motivating support and help of our course coordinator of Object Oriented Programming Through Java, Assistant Professor, Ms. K. Anitha CSE - (AI & ML) & IoT and R&AI. We thank her for the guidance, constant supervision and for providing necessary information to complete this project. Our thanks and appreciations also go to all the faculty members, staff members of VNRVJIET, and all our friends who have helped us put this project together.

BANDI SIDHARTHA REDDY (25071A6679)

BORRA PAVAN SAI ABHISHEK (25071A6683)

NAGOTHU MITHESH (25071A66B5)

SINDARAM SAATHWIK REDDY (25071A66D4)

ABSTRACT

The **Resume Skill Gap Analyser** is a Java-based application designed to help students and job seekers identify the skills they possess and the skills required for a selected job role. In the existing system, users generally compare their resumes manually with job descriptions or use separate online platforms, making it difficult to obtain a simple and structured view of their skill gaps. The proposed system improves this process by providing a centralized application that compares user-provided skills with the required skills of a selected role and displays matching skills, missing skills, and an overall skill-match percentage. The application uses Java concepts such as OOP, interfaces, collections, exception handling, file handling, serialization, multithreading, GUI components, and JDBC for database connectivity and CRUD operations. It provides a simple interface for entering resume details and analysing career-related skill requirements. In future, the system can be extended with NLP-based resume parsing, machine learning-based skill matching, live job-description integration, personalized course recommendations, and progress tracking.

Key Words: Multithreading, JDBC, CRUD, NLP-based Resume Parsing

TABLE OF CONTENTS

1. Problem Statement	**1**
2. Literature Survey	**2**
3. Objectives	**3**
4. System Architecture/ Workflow	**4**
5. Proposed Solution	**5**
6. System Design	**6-10**
7. Implementation	**11-30**
8. Testing and Debugging	**31-34**
9. Analysis of Results	**35**
References	**36**

CHAPTER 1

PROBLEM STATEMENT

With increasing competition for technical jobs, students need to develop skills that match the requirements of their desired job roles. However, many students are uncertain whether their current skills are sufficient for the roles they are interested in. Although job descriptions provide information about required skills, students usually have to manually compare these requirements with the skills mentioned in their resumes. This makes it difficult to clearly identify the skills they already possess and the skills they still need to develop.

The problem becomes more challenging when students consider multiple job roles, as each role may require different programming languages, technologies, tools, databases, and technical competencies. Manual comparison can be time-consuming and may cause students to overlook important requirements. As a result, students may not have a clear understanding of the areas they need to improve.

Therefore, there is a need for a simple system that can analyse a student's existing skills and compare them with the requirements of a selected job role, providing a clear understanding of their current skill profile and areas for improvement.

PROBLEM STATEMENT

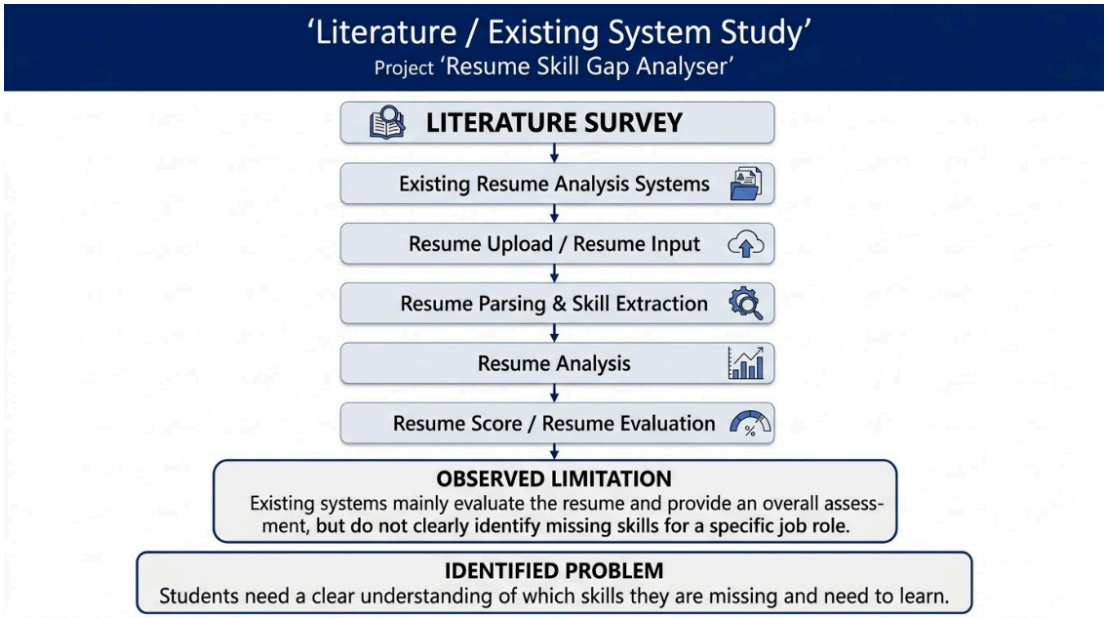
To develop a Java-based Resume Skill Gap Analyser that allows students to compare their existing skills with the skills required for a selected job role. The system will identify matching and missing skills, store job-role and skill-related information in a database, and present the analysis through a user-friendly graphical interface, helping students understand their skill gaps and focus on relevant areas for improvement.

CHAPTER 2
LITERATURE SURVEY

A study of existing resume analysis and career assessment systems was conducted to understand their features and limitations. Most existing systems focus on **analysing resumes, extracting information, identifying keywords, and providing an overall resume score or assessment** based on factors such as skills, experience, education, and resume structure.

These systems are useful for evaluating the quality and relevance of a resume, but they generally focus on **how well the existing resume matches or performs against certain criteria**. They do not always provide a detailed analysis of the **skills that are missing for a specific target job role**. As a result, students may know the current strength or score of their resume but may not clearly understand **what additional skills they need to learn** to become better suited for their desired role.

The study identified a need for a system that goes beyond simply evaluating a resume and provides a **skill-gap analysis** by comparing the student's existing skills with the required skills for a selected job role. This observation forms the basis for developing the proposed **Resume Skill Gap Analyser**.



CHAPTER 3

OBJECTIVES

- **To develop a Java-based system** that analyses a student's resume and existing skills.
- **To compare the student's skills with the required skills** for a selected job role.
- **To identify matching and missing skills** through systematic skill-gap analysis.
- **To store and manage job roles, required skills, and analysis results** using a database.
- **To provide a simple graphical interface** that presents the skill-gap results clearly and helps students identify areas for improvement.

CHAPTER 4

SYSTEM ARCHITECTURE / WORKFLOW

The Resume Skill Gap Analyser follows a modular architecture consisting of a Java Swing-based user interface, skill analysis modules, and an SQLite database. The user provides resume details and selects a target job role for analysis. The application processes the information, retrieves the required skills for the selected job role, and compares them with the existing skills. The system then identifies matching, missing skills and displays the analysis results through the graphical interface. JDBC is used to establish communication between the Java application and the SQLite database.

System Architecture Flow

Student / User

↓

Java Swing User Interface

↓

Input & Resume Processing

↓

Skill Analysis Module

↓

Skill Comparison

↓

Skill Gap Identification

↓

Matching Skills + Missing Skills

↓

Analysis Results

Database Interaction

Java Application ↔ JDBC ↔ SQLite Database

The database maintains information regarding required skills, student skills, and analysis results.

CHAPTER 5
PROPOSED SOLUTION

Existing resume analysis systems primarily focus on **resume evaluation, keyword extraction, and providing an overall resume score**. Although these systems can indicate the quality or relevance of a resume, they may not clearly identify the specific skills a student is missing for a particular job role. As a result, students may find it difficult to understand what additional skills they need to learn or improve.

The proposed solution is a **Resume Skill Gap Analyser** that focuses on identifying the difference between a student's existing skills and the skills required for a selected job role. The system will allow the user to provide their resume details or existing skills and select a target job role. The required skills for that role will be retrieved from the database and compared with the student's available skills.

Based on the comparison, the system will identify **matching skills and missing skills** and present the results in a simple and understandable format. This will provide students with a clearer view of their current skill profile and the areas where improvement is required. The system will also maintain job roles and their corresponding skill requirements in a database, allowing the analysis to be performed for different target roles.

The application will be developed using **Java and Java Swing**, with **SQLite** used for data storage and **JDBC** for database connectivity. By combining these technologies, the proposed system provides a structured and user-friendly approach to analysing skill gaps and helping students understand the skills they need to develop for their desired job roles.

CHAPTER 6
SYSTEM DESIGN

The system is designed as a Java-based application that analyzes a user's resume against the skills required for a selected job role. It is divided into multiple modules that handle resume input, skill extraction, skill comparison, gap identification, and result storage.

Main Modules

1. User Interface Module
   - Provides screens for entering resume details and selecting a target job role.
   - Displays the analysis results in a simple and understandable format.
2. Resume Analysis Module
   - Processes the entered resume information.
   - Identifies the technical and relevant skills mentioned in the resume.
3. Skill Matching Module
   - Compares extracted resume skills with required skills for the selected job role.
   - Categorizes skills into Matched Skills and Missing Skills.
4. Skill Gap Analysis Module
   - Determines the skills that the user needs to improve or acquire.
   - Generates a skill-gap summary based on the comparison.
5. Database Module
   - Stores user details, job roles, required skills, resume analysis results, and identified skill gaps using SQLite.
6. Result & Recommendation Module
   - Presents the final analysis to the user.
   - Provides list of missing skills that can be considered for further learning.

Core Algorithm

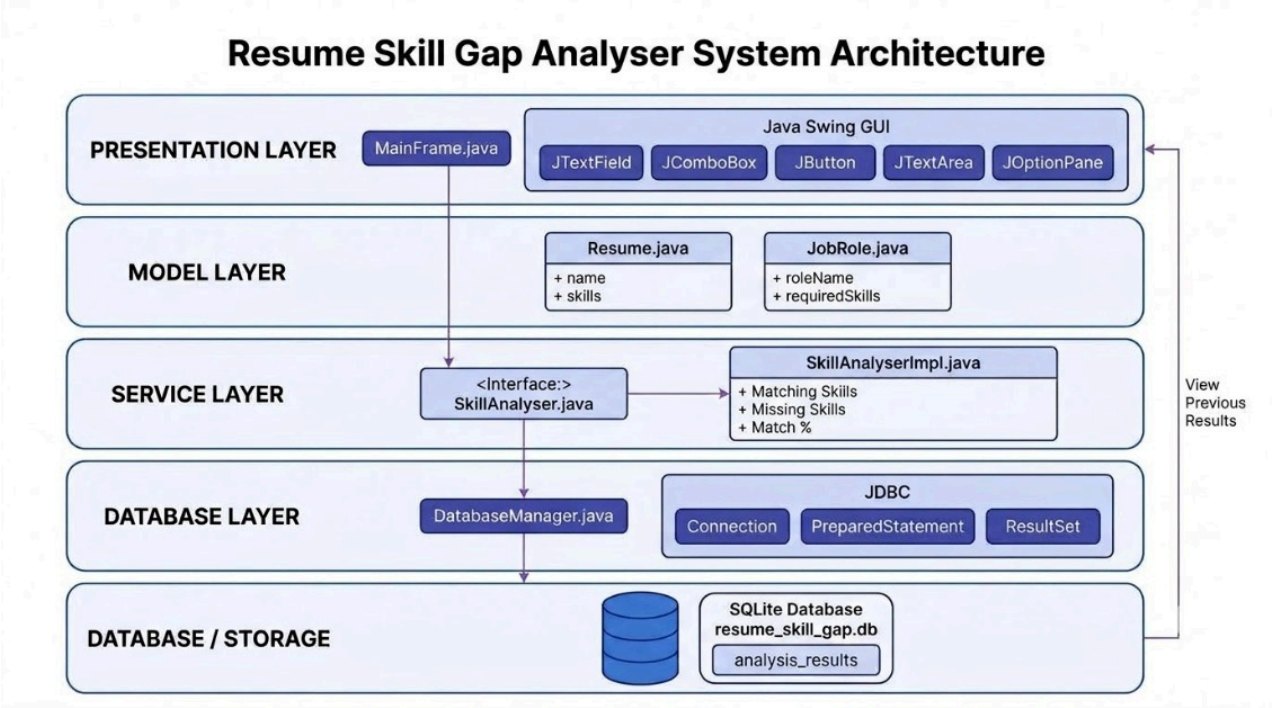
Resume Input → Skill Extraction → Job Role Selection → Required Skill Retrieval → Skill Comparison → Gap Identification → Result Display → Database Storage

Design Diagrams

System Architecture Diagram:

The system follows a simple layered architecture consisting of the Presentation Layer, Application/Analysis Layer, and Data Layer.

- Presentation Layer: Provides the Java Swing-based graphical user interface through which users enter resume details and select a target job role.
- Application Layer: Processes the resume, extracts available skills, compares them with the required skills, and identifies skill gaps.
- Data Layer: Uses SQLite to store user information, job roles, skills, and analysis results.



Functional Modules

The system is divided into the following functional modules:

3.1 User Interface Module

Provides the interface for entering resume information, selecting a target job role, initiating analysis, and viewing the generated results.

3.2 Resume Processing Module

Processes the resume information provided by the user and identifies the skills present in the resume.

3.3 Job Requirement Module

Maintains the required skills associated with different job roles and provides the relevant skill requirements for comparison.

3.4 Skill Matching Module

Compares the skills identified from the resume with the required skills of the selected job role.

3.5 Skill Gap Analysis Module

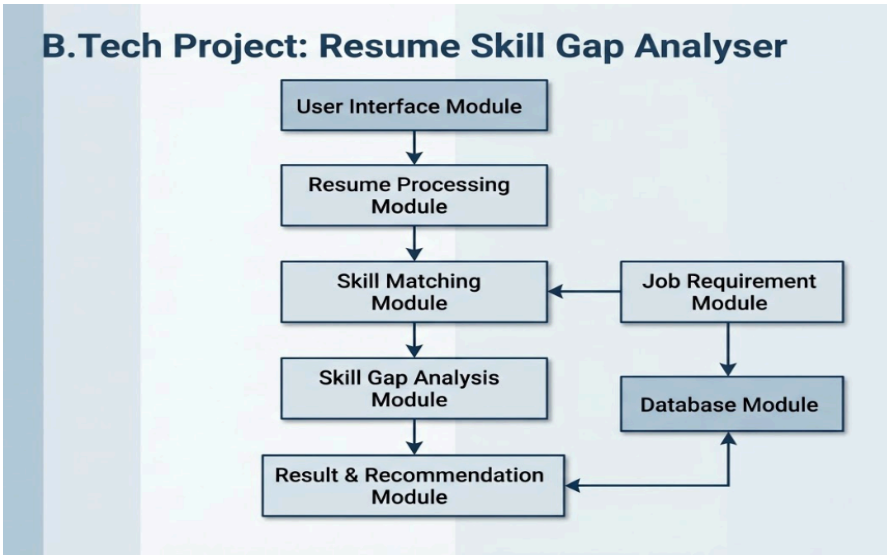
Identifies missing skills by determining which required skills are not present in the user's resume.

3.6 Database Module

Stores and retrieves user information, job roles, skills, and analysis results using SQLite.

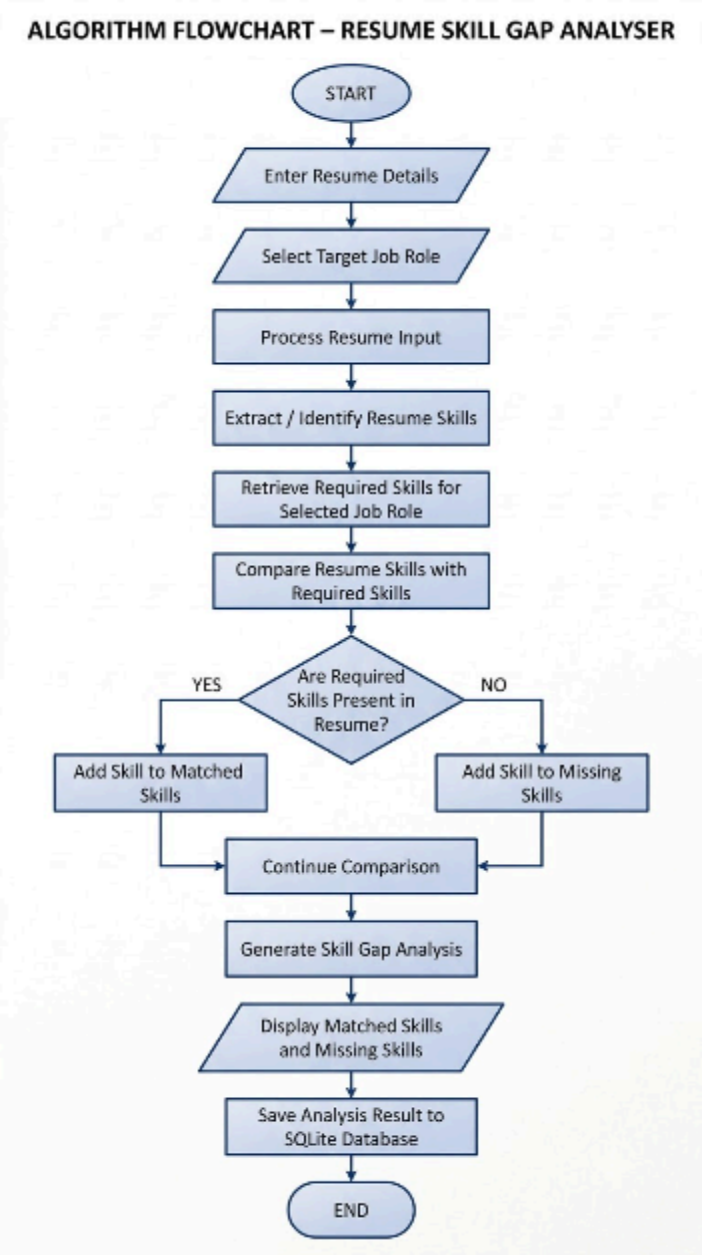
3.7 Result Display Module

Displays matched skills, missing skills, and the overall skill-gap analysis to the user.



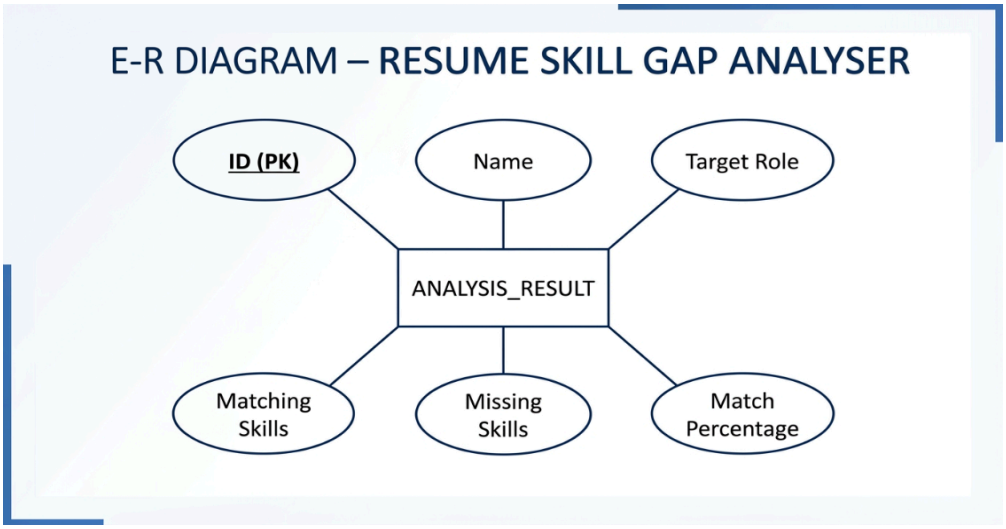
Algorithm Design

The algorithm first obtains the user's skills and the required job skills. It then compares required skills with the user's skill set. Skills found in both sets are classified as matched skills, while skills that are required but not found in the resume are classified as missing skills. The resulting information is then displayed to the user and stored in the database.



Database Design

SQLite is used as the database management system for storing and retrieving the information required by the application. The database is designed to maintain user details, job-role requirements, skills, and generated analysis results. The database enables previously generated analysis information to be stored systematically and retrieved when required.



Database Schema

Attribute	**Data Type**	**Key Constraint**	**Description**
ID	INTEGER	Primary Key	Unique ID for each analysis record
Name	TEXT	Not Null	Name of the user
TargetRole	TEXT	Not Null	Job role selected by the user
MatchingSkills	TEXT	—	Skills matched with the requirements of the target role
Missing Skills	TEXT	—	Skills required but missing from the resume
Match Percentage	REAL	Check(>=0)	Percentage of skills matched

CHAPTER 7

IMPLEMENTATION

1. Database Connection

This module establishes a connection between the Java application and the SQLite database using JDBC.

Code:

```
package skillgap.database;


import java.io.File;

import java.sql.Connection;

import java.sql.DriverManager;

import java.sql.PreparedStatement;

import java.sql.ResultSet;

import java.util.List;


public class DatabaseManager {

    private static final String DB_FILE = "resume_skill_gap.db";

    private static final String URL = "jdbc:sqlite:" + DB_FILE;

     public DatabaseManager() {

        try {

            Class.forName("org.sqlite.JDBC");
```

```
            System.out.println("SQLite driver loaded successfully.");
            System.out.println("Database location: " + new
File(DB_FILE).getAbsolutePath());
            createTable();
        } catch (Exception e) {
            System.out.println(
                    "Database initialization failed:"
            );
            e.printStackTrace();
        }
    }
    private Connection connect() throws Exception {
        return DriverManager.getConnection(URL);
    }
    private void createTable() throws Exception {
        String sql =
                "CREATE TABLE IF NOT EXISTS analysis_results (" +
                "id INTEGER PRIMARY KEY AUTOINCREMENT, " +
                "name TEXT NOT NULL, " +
                "target_role TEXT NOT NULL, " +
                "matching_skills TEXT, " +
                "missing_skills TEXT, " +
                "match_percentage REAL)";
```

```
    try (Connection connection = connect();
         PreparedStatement statement =
                 connection.prepareStatement(sql)) {
        statement.executeUpdate();
        System.out.println(
                "Database table created/verified successfully."
        );
    }
}
public boolean saveResult(
        String name,
        String targetRole,
        List<String> matchingSkills,
        List<String> missingSkills,
        double matchPercentage) {
    String sql =
            "INSERT INTO analysis_results " +
            "(name, target_role, matching_skills, " +
            "missing_skills, match_percentage) " +
            "VALUES (?, ?, ?, ?, ?)";
```

```
        try (Connection connection = connect();
             PreparedStatement statement =
                     connection.prepareStatement(sql)) {
            statement.setString(1, name);
            statement.setString(2, targetRole);
            statement.setString(
                    3,
                    String.join(", ", matchingSkills)
            );
            statement.setString(
                    4,
                    String.join(", ", missingSkills)
            );
            statement.setDouble(
                    5,
                    matchPercentage
            );
            statement.executeUpdate();
            System.out.println("Analysis result saved successfully.");
            return true;
        } catch (Exception e) {
            System.out.println("ERROR: Could not save analysis result.");
```

```
            e.printStackTrace();
            return false;
        }
    }
    public String getPreviousResults() {
        StringBuilder results = new StringBuilder();
        String sql =
                "SELECT * FROM analysis_results " +
                "ORDER BY id DESC";
        try (Connection connection = connect();
             PreparedStatement statement =
                     connection.prepareStatement(sql);
             ResultSet resultSet =
                     statement.executeQuery()) {
            while (resultSet.next()) {
                results.append("ID: ")
                        .append(resultSet.getInt("id"))
                        .append("\n");
                results.append("Name: ")
                        .append(resultSet.getString("name"))
                        .append("\n");
```

```
            results.append("Target Role: ")
                    .append(resultSet.getString("target_role"))
                    .append("\n");
            results.append("Matching Skills: ")
                    .append(resultSet.getString(
                            "matching_skills"))
                    .append("\n");
            results.append("Missing Skills: ")
                    .append(resultSet.getString(
                            "missing_skills"))
                    .append("\n");
            results.append("Match Percentage: ")
                    .append(String.format("%.2f",
                            resultSet.getDouble("match_percentage")))
                    .append("%\n");
            results.append("----------------------------\n");
        }
    } catch (Exception e) {
        e.printStackTrace();
        return "Database error: " + e.getMessage();
    }
```

```
        if (results.length() == 0) {
            return "No previous results found.";
        }
        return results.toString();
    }
}
```

2. Skill Gap Analysis

This module contains the student's existing skills and the required skills for the selected job role.

Job Role Code:

```
package skillgap.model;
import java.util.List;
public class JobRole {
    private String roleName;

    private List<String> requiredSkills;

    public JobRole(String roleName, List<String> requiredSkills) {

        this.roleName = roleName;

        this.requiredSkills = requiredSkills;

    }

    public String getRoleName() {

        return roleName;

    }

    public List<String> getRequiredSkills() {

        return requiredSkills;

    }

}
```

Resume File Code:

```
package skillgap.model;
import java.util.ArrayList;
import java.util.List;
public class Resume {
    private String name;
    private List<String> skills;
    public Resume() {
        skills = new ArrayList<>();
    }

    public Resume(String name, List<String> skills) {
        this.name = name;
        this.skills = skills;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public List<String> getSkills() {
        return skills;
    }

    public void setSkills(List<String> skills) {
        this.skills = skills;
    }
}
```

3. Analysis Result

The identified matching and missing skills are displayed to the user through the Java Swing interface.

Skill Analyser Code:

```
package skillgap.service;

import skillgap.model.Resume;

import skillgap.model.JobRole;

import java.util.List;

public interface SkillAnalyser {

    List<String> findMissingSkills(

            Resume resume,

            JobRole jobRole

    );

    List<String> findMatchingSkills(

            Resume resume,

            JobRole jobRole

    );

    double calculateMatchPercentage(

            Resume resume,

            JobRole jobRole

    );

}
```

Skill Gap Analysis Code:

```
package skillgap.service;
import skillgap.model.Resume;
import skillgap.model.JobRole;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
public class SkillAnalyserImpl implements SkillAnalyser {
   public List<String> findMissingSkills(
         Resume resume,
         JobRole jobRole) {

      Set<String> userSkills = new HashSet<>();

      for (String skill : resume.getSkills()) {
         userSkills.add(skill.trim().toLowerCase());
      }

      List<String> missingSkills = new ArrayList<>();

      for (String skill : jobRole.getRequiredSkills()) {

         if (!userSkills.contains(
               skill.trim().toLowerCase())) {

            missingSkills.add(skill);
         }
      }

      return missingSkills;
   }
```

```
    public List<String> findMatchingSkills(
            Resume resume,
            JobRole jobRole) {

        Set<String> userSkills = new HashSet<>();

        for (String skill : resume.getSkills()) {
            userSkills.add(skill.trim().toLowerCase());
        }
        List<String> matchingSkills = new ArrayList<>();

        for (String skill : jobRole.getRequiredSkills()) {

            if (userSkills.contains(
                    skill.trim().toLowerCase())) {

                matchingSkills.add(skill);
            }
        }
        return matchingSkills;
    }
    public double calculateMatchPercentage(
            Resume resume,
            JobRole jobRole) {

        if (jobRole.getRequiredSkills().isEmpty()) {
            return 0;
        }

        List<String> matchingSkills =
                findMatchingSkills(resume, jobRole);

        return (matchingSkills.size() * 100.0)/ jobRole.getRequiredSkills().size();
    }
}
```

4. GUI Implementation

The Java Swing interface provides input fields for student skills and job-role selection and displays the generated skill-gap analysis

Code:

```
package skillgap.gui;
import skillgap.database.DatabaseManager;
import skillgap.model.JobRole;
import skillgap.model.Resume;
import skillgap.service.SkillAnalyser;
import skillgap.service.SkillAnalyserImpl;
import javax.swing.*;
import java.awt.*;
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
public class MainFrame extends JFrame {
    private JTextField nameField;
    private JTextField skillsField;
    private JComboBox<String> roleBox;
    private JTextArea resultArea;
    private DatabaseManager databaseManager;
    public MainFrame() {
        setTitle("Resume Skill Gap Analyser");
        setSize(650, 550);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);

        databaseManager = new DatabaseManager();

        JPanel panel = new JPanel(new GridLayout(6, 2, 10, 10));
```

```
JLabel nameLabel = new JLabel("Name:");
nameField = new JTextField();

JLabel skillsLabel = new JLabel("Skills:");
skillsField = new JTextField();
JLabel roleLabel = new JLabel("Target Job Role:");
String[] roles = {
        "Java Developer",
        "Data Analyst",
        "Web Developer",
        "Software Developer"
};
roleBox = new JComboBox<>(roles);
JButton analyseButton = new JButton("Analyse Resume");
JButton historyButton = new JButton("View Previous Results");

resultArea = new JTextArea();
resultArea.setEditable(false);

panel.add(nameLabel);
panel.add(nameField);

panel.add(skillsLabel);
panel.add(skillsField);

panel.add(roleLabel);
panel.add(roleBox);

panel.add(new JLabel());
panel.add(analyseButton);

panel.add(new JLabel());
panel.add(historyButton);

add(panel, BorderLayout.NORTH);
```

```
        add(new JScrollPane(resultArea),
                BorderLayout.CENTER);

        analyseButton.addActionListener(
                e -> analyseResume()
        );

        historyButton.addActionListener(
                e -> showPreviousResults()
        );
    }

    private void analyseResume() {

        try {

            String name = nameField.getText().trim();

            String skillText = skillsField.getText().trim();

            if (name.isEmpty() ||  skillText.isEmpty()) {
                throw new IllegalArgumentException(
                        "Please enter name and skills."
                );
            }

             List<String> userSkills = new ArrayList<>( Arrays.asList( skillText.split(",")
) );

            Resume resume = new Resume(name, userSkills);

            JobRole jobRole = createJobRole(roleBox.getSelectedItem().toString());

            SkillAnalyser analyser = new SkillAnalyserImpl();
```

```
List<String> matching = analyser.findMatchingSkills(resume,jobRole);

List<String> missing = analyser.findMissingSkills(resume,jobRole);

double percentage = analyser.calculateMatchPercentage(resume,jobRole);

StringBuilder result = new StringBuilder();

result.append("RESUME SKILL GAP ANALYSIS\n");

result.append("==========================\n\n");

result.append("Name: ")
      .append(name)
      .append("\n");

result.append("Target Role: ")
      .append(jobRole.getRoleName())
      .append("\n\n");

result.append("Matching Skills:\n");

if (matching.isEmpty()) {
   result.append("None\n");
} else {
   for (String skill : matching) {
      result.append("- ")
            .append(skill)
            .append("\n");
   }
}

result.append("\nMissing Skills:\n");

if (missing.isEmpty()) {
```

```
            result.append("None\n");
        } else {
            for (String skill : missing) {

                result.append("- ")
                      .append(skill)
                      .append("\n");
            }
        }

        result.append("\nSkill Match Percentage: ")
              .append(String.format("%.2f",percentage))
              .append("%");

        resultArea.setText(result.toString());

        boolean saved = databaseManager.saveResult(
              name,
              jobRole.getRoleName(),
              matching,
              missing,
              percentage
        );

        if (saved) {
            JOptionPane.showMessageDialog(
                  this,
                  "Analysis completed and saved.",
                  "Success",
                  JOptionPane.INFORMATION_MESSAGE
            );
        } else {
            JOptionPane.showMessageDialog(
                  this,
                  "Analysis completed, but database saving failed. " +
```

```
                "Check the Eclipse Console.",
                "Database Error",
                JOptionPane.ERROR_MESSAGE
            );
        }
    } catch (IllegalArgumentException ex) {

        JOptionPane.showMessageDialog(
            this,
            ex.getMessage(),
            "Input Error",
            JOptionPane.ERROR_MESSAGE
        );
    }
}
private void showPreviousResults() {
    String previousResults =
        databaseManager.getPreviousResults();
    resultArea.setText(
        "PREVIOUS ANALYSIS RESULTS\n" +
        "============================\n\n" +
        previousResults
    );
}

private JobRole createJobRole(String role) {

    List<String> skills = new ArrayList<>();
    switch (role) {
        case "Java Developer":

            skills.addAll(
                Arrays.asList(
                    "Java",
                    "OOP",
```

```
                "SQL",
                "JDBC",
                "Git",
                "Spring"
            )
        );

        break;

    case "Data Analyst":

        skills.addAll(
            Arrays.asList(
                "Python",
                "SQL",
                "Excel",
                "Statistics",
                "Power BI"
            )
        );

        break;

    case "Web Developer":

        skills.addAll(
            Arrays.asList(
                "HTML",
                "CSS",
                "JavaScript",
                "SQL",
                "Git"
            )
        );
```

```
                break;

            case "Software Developer":

                skills.addAll(
                    Arrays.asList(
                        "Java",
                        "C++",
                        "OOP",
                        "SQL",
                        "Git"
                    )
                );

                break;
            }
            return new JobRole(
                role,
                skills
            );
        }
        public static void main(String[] args) {

            SwingUtilities.invokeLater(
                () -> {

                    MainFrame frame =
                        new MainFrame();

                    frame.setVisible(true);
                }
            );
        }
    }
```

CHAPTER 8

TESTING AND DEBUGGING

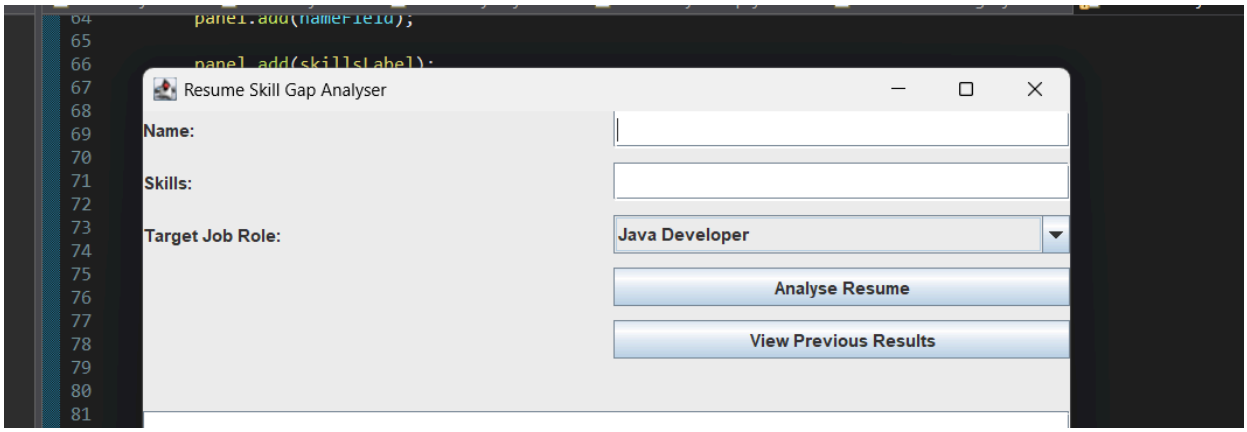
Testing Objectives

- To verify that the application accepts valid user inputs.
- To ensure that candidate skills are compared correctly with job requirements.
- To check whether missing skills are identified accurately.
- To verify the calculation of the skill-match percentage.
- To ensure that invalid or empty inputs are handled appropriately.
- To confirm that analysis results are displayed correctly.

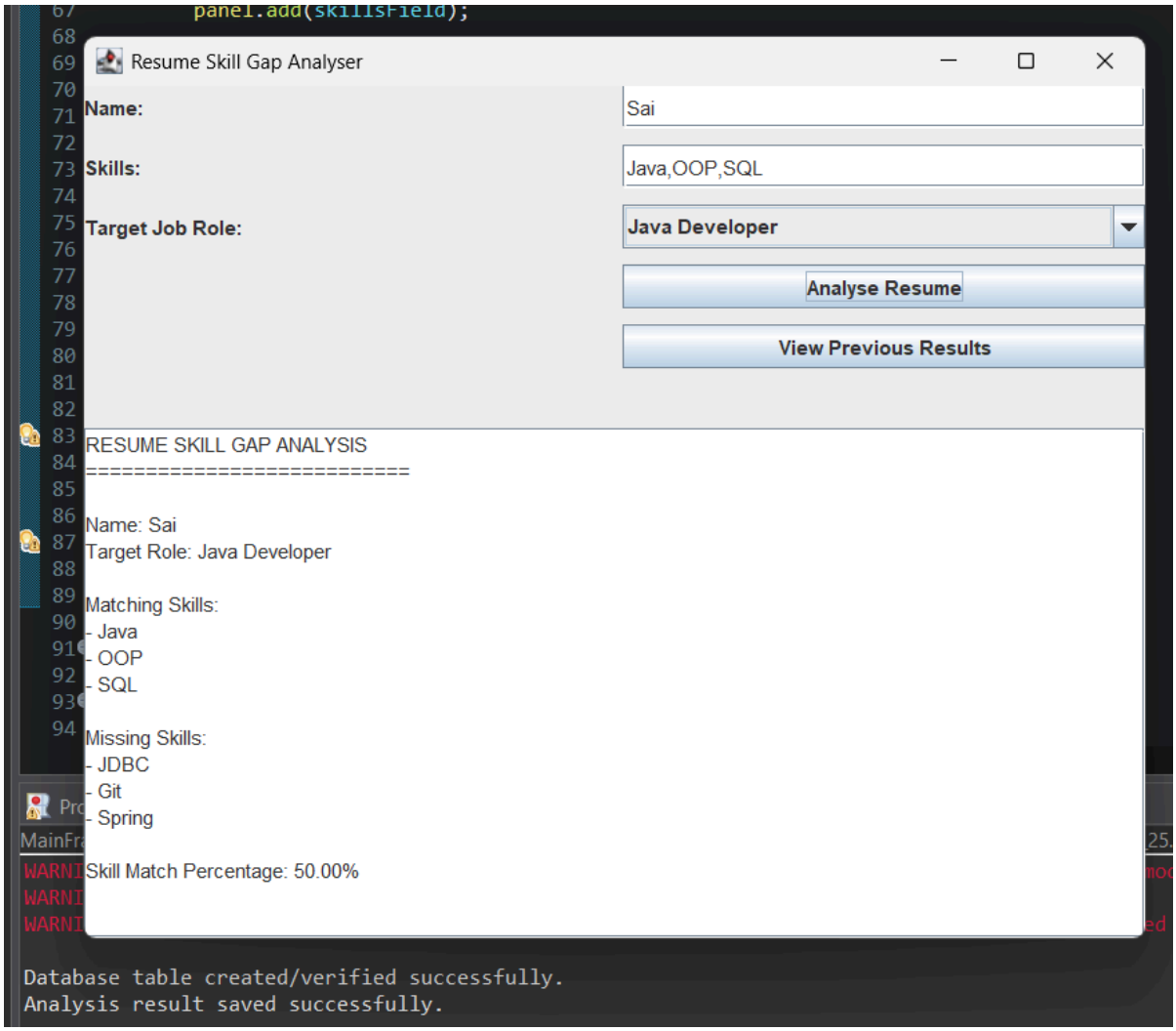
Test Case ID	Test Scenario	Expected Result	Status
TC01	Enter valid candidate skills and select a job role	Analysis is performed successfully	Pass
TC02	Enter skills that match all required job skills	100% skill match is displayed	Pass
TC03	Enter only some of the required skills	Matched and missing skills are displayed correctly	Pass
TC04	Enter skills that do not match the selected job role	Missing skills are identified and the match percentage is calculated	Pass
TC05	Submit the form without entering required information	Appropriate validation message is displayed	Pass
TC06	Enter multiple skills separated by commas	Individual skills are processed correctly	Pass
TC07	Enter skills that are completely unrelated to the selected job role	No matching skills are found, all required skills are listed as missing, and the match percentage is displayed as 0%	Pass

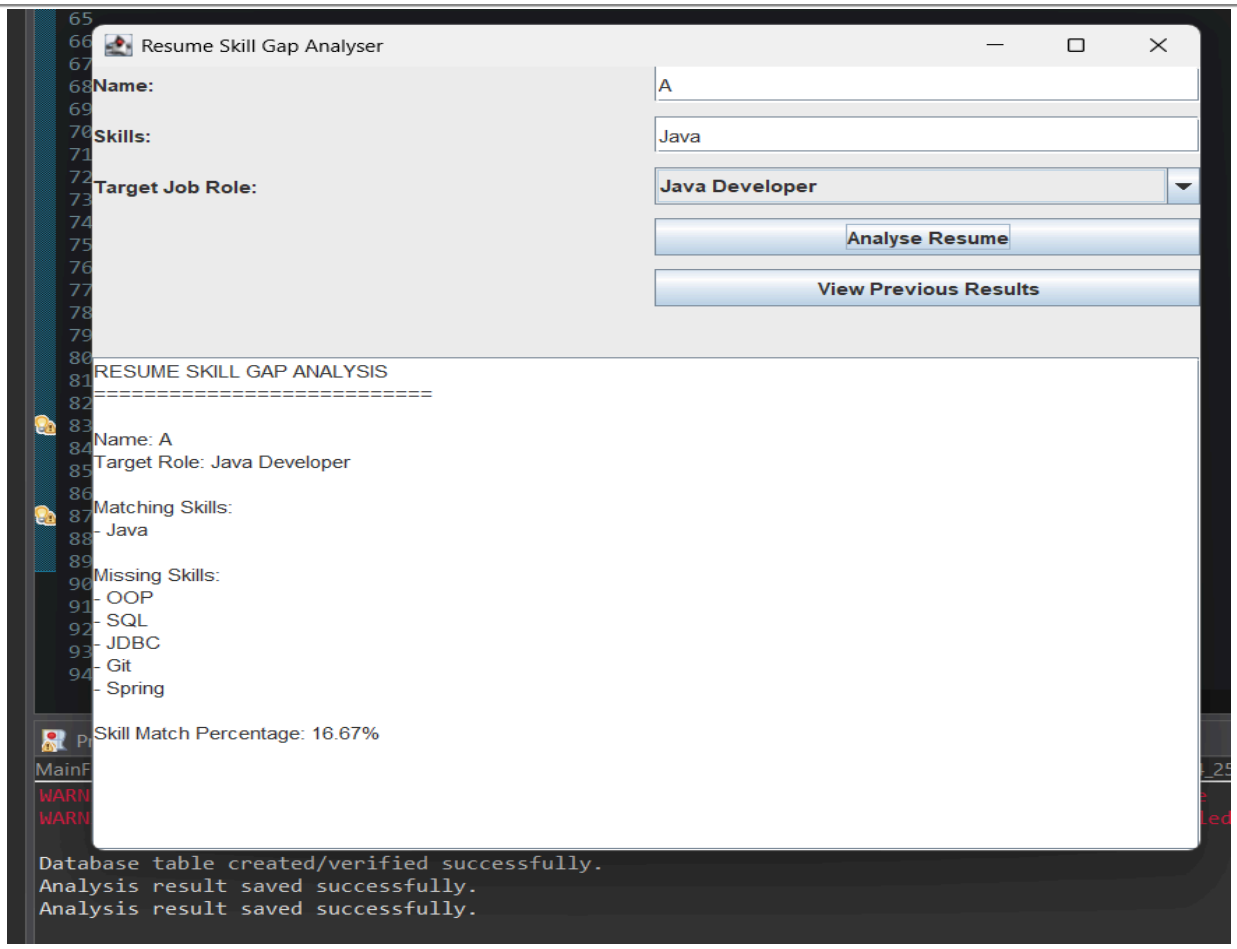
The test cases are designed to verify the major functionalities of the Resume Skill Gap Analyser. They cover input validation, skill comparison, missing-skill identification, and match-percentage calculation. The results help ensure that the application provides meaningful feedback to candidates about the skills they possess and the skills they need to develop for their selected job roles.

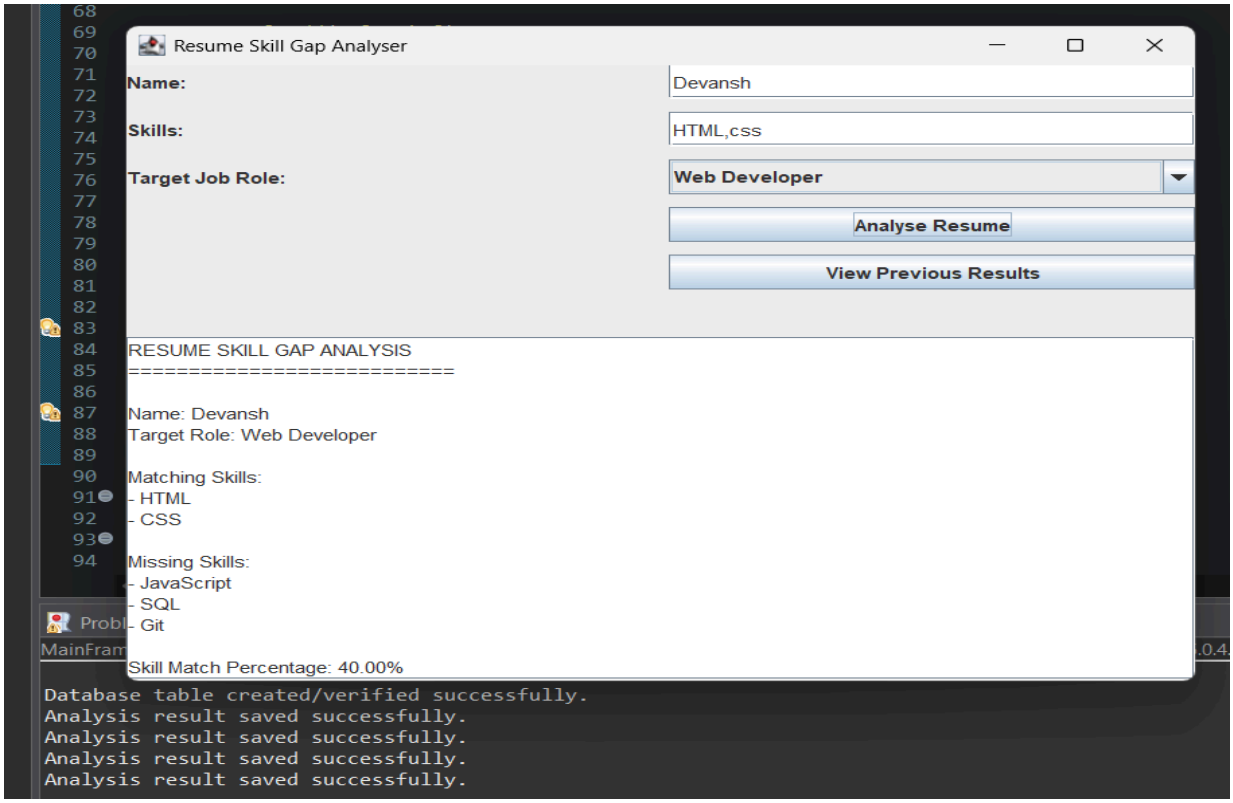
The Graphical User Interface (GUI) during Testing:



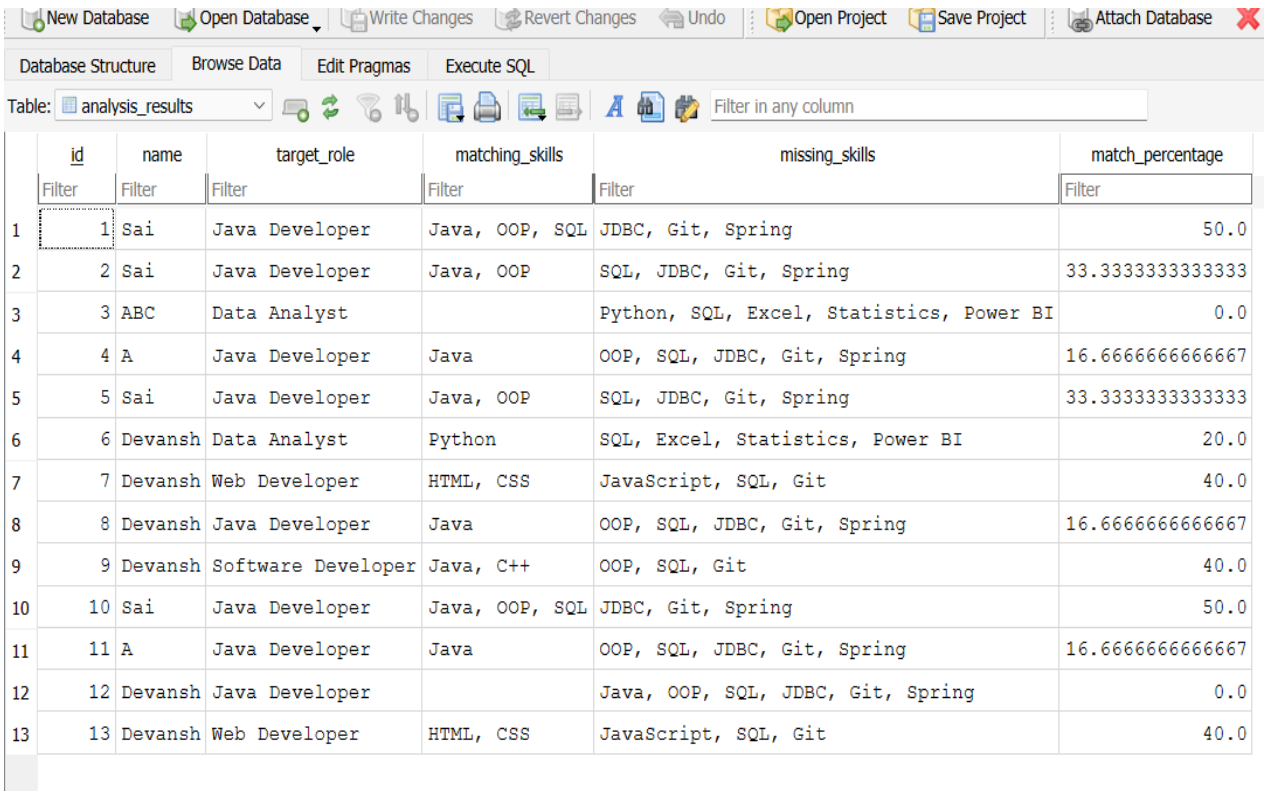
Few Random test cases were added to test the working of the Application:




Resume Skill Gap Analyser
Name:
A
Skills:
Java
Target Job Role:
Java Developer
Analyse Resume
View Previous Results
RESUME SKILL GAP ANALYSIS
==========
Name: A
Target Role: Java Developer
Matching Skills:
- Java
Missing Skills:
- OOP
- SQL
- JDBC
- Git
- Spring
Skill Match Percentage: 16.67%
Database table created/verified successfully.
Analysis result saved successfully.
Analysis result saved successfully.

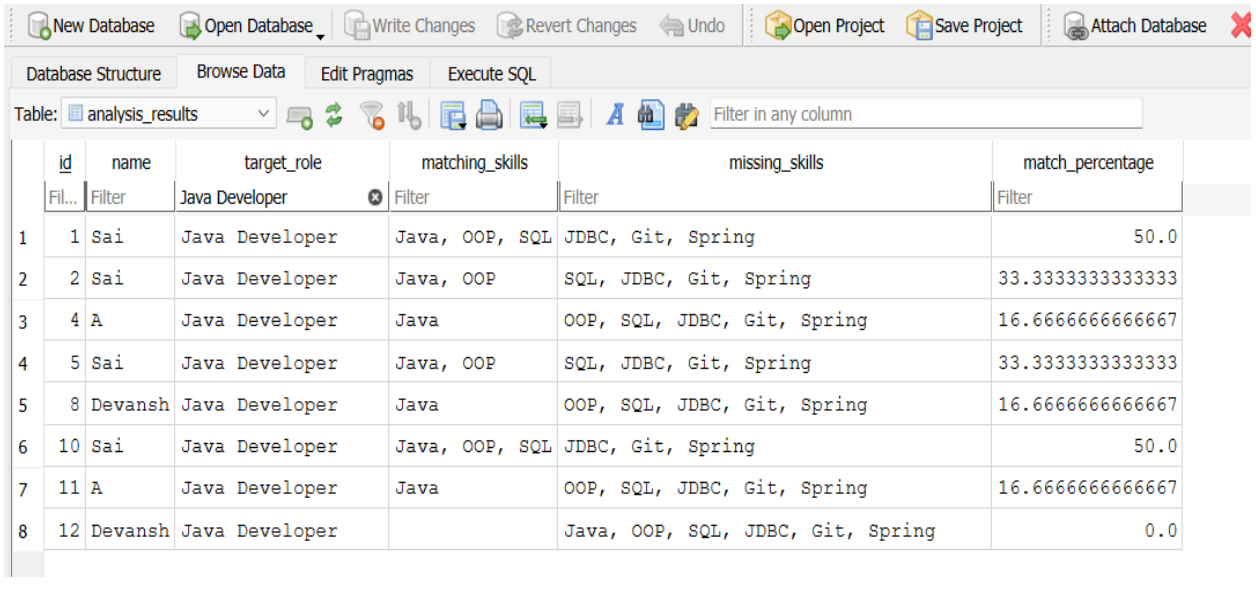

Resume Skill Gap Analyser
Name:
Devansh
Skills:
HTML,css
Target Job Role:
Web Developer
Analyse Resume
View Previous Results
RESUME SKILL GAP ANALYSIS
==========
Name: Devansh
Target Role: Web Developer
Matching Skills:
- HTML
- CSS
Missing Skills:
- JavaScript
- SQL
- Git
Skill Match Percentage: 40.00%
Database table created/verified successfully.
Analysis result saved successfully.
Analysis result saved successfully.
Analysis result saved successfully.
Analysis result saved successfully.

The Database after sample testing is done is:



	id	name	target_role	matching_skills	missing_skills	match_percentage
1	1	Sai	Java Developer	Java, OOP, SQL	JDBC, Git, Spring	50.0
2	2	Sai	Java Developer	Java, OOP	SQL, JDBC, Git, Spring	33.3333333333333
3	3	ABC	Data Analyst		Python, SQL, Excel, Statistics, Power BI	0.0
4	4	A	Java Developer	Java	OOP, SQL, JDBC, Git, Spring	16.6666666666667
5	5	Sai	Java Developer	Java, OOP	SQL, JDBC, Git, Spring	33.3333333333333
6	6	Devansh	Data Analyst	Python	SQL, Excel, Statistics, Power BI	20.0
7	7	Devansh	Web Developer	HTML, CSS	JavaScript, SQL, Git	40.0
8	8	Devansh	Java Developer	Java	OOP, SQL, JDBC, Git, Spring	16.6666666666667
9	9	Devansh	Software Developer	Java, C++	OOP, SQL, Git	40.0
10	10	Sai	Java Developer	Java, OOP, SQL	JDBC, Git, Spring	50.0
11	11	A	Java Developer	Java	OOP, SQL, JDBC, Git, Spring	16.6666666666667
12	12	Devansh	Java Developer		Java, OOP, SQL, JDBC, Git, Spring	0.0
13	13	Devansh	Web Developer	HTML, CSS	JavaScript, SQL, Git	40.0

Filters can be applied on the fields to retrieve data easily:



	id	name	target_role	matching_skills	missing_skills	match_percentage
1	1	Sai	Java Developer	Java, OOP, SQL	JDBC, Git, Spring	50.0
2	2	Sai	Java Developer	Java, OOP	SQL, JDBC, Git, Spring	33.3333333333333
3	4	A	Java Developer	Java	OOP, SQL, JDBC, Git, Spring	16.6666666666667
4	5	Sai	Java Developer	Java, OOP	SQL, JDBC, Git, Spring	33.3333333333333
5	8	Devansh	Java Developer	Java	OOP, SQL, JDBC, Git, Spring	16.6666666666667
6	10	Sai	Java Developer	Java, OOP, SQL	JDBC, Git, Spring	50.0
7	11	A	Java Developer	Java	OOP, SQL, JDBC, Git, Spring	16.6666666666667
8	12	Devansh	Java Developer		Java, OOP, SQL, JDBC, Git, Spring	0.0

CHAPTER 9

ANALYSIS OF RESULTS

Conclusion

The Resume Skill Gap Analyser was developed as a Java-based application to help users understand how their existing skills match the requirements of a selected job role. Users can enter their name and skills, select a target job role, and view their matching and missing skills along with a skill-match percentage. The results can also be saved and previous analysis results can be viewed.

The project helped us apply Java concepts such as classes, objects, constructors, interfaces, collections, exception handling, and Swing GUI components in a practical application. The use of separate classes such as Resume, JobRole, and SkillAnalyser helped us understand modular application development.

Future Scope

The current Resume Skill Gap Analyser provides a foundation for analysing and storing a user's skill information. The system can be further enhanced with the following features:

- Resume File Upload: Allow users to upload resumes in PDF or DOCX format instead of entering skills manually.
- Expanded Job Roles: Include a larger set of job roles with regularly updated skill requirements.
- Personalized Recommendations: Suggest courses, certifications, and learning resources based on the user's missing skills.
- Skill Progress Tracking: Allow users to update their skills and monitor their improvement over time.
- Advanced Skill Matching: Use machine learning techniques to identify related or similar skills instead of relying only on exact skill matches.

These enhancements can make the application more automated, personalized, and useful for students and job seekers while building upon the existing Java and JDBC-based system.

REFERENCES

1. Oracle. *Java Documentation – Java Platform, Standard Edition (Java SE).*
   Java SE Documentation
2. Oracle. *The Java Tutorials – Object-Oriented Programming Concepts.*
   Object-Oriented Programming Concepts
3. Oracle. *The Java Tutorials – Collections.*
   Java Collections Framework
4. Oracle. *The Java Tutorials – Exceptions.*
   Java Exception Handling
5. Oracle. *The Java Tutorials – Creating a GUI With Swing.*
   Java Swing Tutorial
6. Oracle. *JDBC Basics.*
   JDBC Basics
7. MySQL. *MySQL Connector/J Developer Guide.*
   MySQL Connector/J Documentation